

C++ OOP Practice Problems

A structured set of coding problems to practice Object-Oriented Programming in C++

How to use this sheet	Solve in order. Focus on designing classes first, then write main(). Avoid putting all logic inside main().
Core concepts	Classes & objects, constructors, encapsulation, inheritance, polymorphism, abstraction, composition, and validation.
Rule	For the first few problems, try without looking at solutions. Treat each problem as a small interview-style exercise.

LEVEL 1 - BASIC OOP

1. Student Management

Create a Student class with name, rollNo, and marks in 3 subjects. Add calculatePercentage() and displayDetails(). Create objects for 3 students and display their details.

Concepts: class, object, constructors, member functions.

2. Bank Account

Create a BankAccount class containing account holder name, account number, and balance. Add deposit(amount), withdraw(amount), and displayBalance(). Do not allow withdrawal when the balance is insufficient.

Concepts: encapsulation, constructors, member functions.

3. Rectangle

Create a Rectangle class with length and width. Add area(), perimeter(), and display(). Create multiple rectangle objects and compare their areas.

Concepts: class design, objects, methods.

LEVEL 2 - CONSTRUCTORS + ENCAPSULATION

4. Employee Salary System

Create an Employee class with employee ID, name, and basic salary. Calculate HRA = 20% of basic, DA = 10% of basic, and gross salary. Keep data members private.

Concepts: encapsulation, constructors, getters/setters, calculations.

5. Mobile Phone

Create a Mobile class with brand, model, price, and battery percentage. Add call(), charge(), and displayInfo(). Ensure battery stays between 0 and 100.

Concepts: data validation, encapsulation, member functions.

6. Library Book

Create a Book class with title, author, price, and isIssued. Add issueBook(), returnBook(), and displayBook(). A book already issued should not be issued again.

Concepts: state management, validation, class behavior.

LEVEL 3 - INHERITANCE

7. Vehicle System

Create a base Vehicle class with brand and speed. Derive Car, Bike, and Truck. Give each class its own display() behavior.

Concepts: inheritance, overriding, base/derived classes.

8. Employee Hierarchy

Create Employee as a base class and derive Manager, Developer, and Intern. Each type calculates salary differently: Manager gets bonus, Developer gets project allowance, Intern gets a fixed stipend.

Concepts: inheritance, overriding, reuse.

LEVEL 4 - POLYMORPHISM

9. Shape Calculator

Create a base Shape class with virtual double area(). Derive Circle, Rectangle, and Triangle and override area(). Store objects using base-class pointers and calculate areas polymorphically.

Concepts: virtual functions, runtime polymorphism, base pointers.

10. Payment System

Create Payment as a base class with makePayment(amount). Derive CreditCard, UPI, and Cash. Each payment type should implement the function differently.

Concepts: runtime polymorphism, overriding, abstraction.

LEVEL 5 - REAL OOP PRACTICE

11. ATM System

Build a mini ATM with Check Balance, Deposit, Withdraw, Change PIN, and Exit. Create an Account class and keep sensitive information private. Handle invalid operations.

Concepts: encapsulation, validation, class responsibilities.

12. Shopping Cart

Create Product, Cart, and Customer classes. Product has id, name, price, quantity. Cart supports addProduct(), removeProduct(), calculateTotal(), and displayCart(). Let Customer own a Cart.

Concepts: composition, has-a relationships, class collaboration.

CHALLENGES

13. Parking Lot System

Design Vehicle, Car, Bike, Truck, ParkingSlot, and ParkingLot. Support parking/removal, available slots, vehicle lookup, and time-based parking fees. Keep logic distributed across classes instead of putting everything in main().

Concepts: inheritance, composition, polymorphism, system design.

14. Online Food Ordering System

Create User, Restaurant, FoodItem, Order, and Payment classes. Support registration, menu display, placing/cancelling orders, bill calculation, payment, and order history. Use OOP concepts where they naturally fit.

Concepts: encapsulation, constructors, inheritance, polymorphism, composition, abstraction.

Recommended Order

1 -> 2 -> 4 -> 6 -> 7 -> 8 -> 9 -> 10 -> 11 -> 12 -> 13 -> 14

Finish Problem 1 first and review your own class design before moving ahead.